

이동형

DevOps Engineer | 경력기술서

핵심 역량 (Core Competencies)

클라우드 인프라 설계 및 운영

AWS, GCP 기반 멀티·하이브리드 클라우드 아키텍처 설계 및 운영 경험

- AWS EKS, GCP GKE 프로덕션 환경 운영 및 비용 최적화 (하이브리드 전환 20% 절감, GCP 마이그레이션 50% 절감, 절감분 신규 프로젝트 재투자)
- Terraform을 활용한 IaC 기반 인프라 자동화 및 버전 관리 (GCP 리소스 100% 코드화)
- 온프레미스-클라우드 하이브리드 아키텍처 설계 및 마이그레이션 경험

CI/CD 파이프라인 구축 및 GitOps

GitOps 기반 배포 자동화로 개발 생산성 극대화

- GitHub Actions, GitLab CI, ArgoCD를 활용한 CI/CD 파이프라인 설계 및 고도화
- 배포 시간 50~67% 단축, GitOps 지속 배포 전환으로 Time to Market(시장 대응 속도) 단축, 롤백 시간 95% 감소 (30분 → 1~2분)
- Jenkins 기반 Unity Android/iOS 빌드 자동화 및 Slack 연동 배포 알림 구축

모니터링 및 오피서빌리티 시스템 구축

장애 사전 감지 및 신속한 대응 체계 수립

- PLG Stack (Prometheus, Loki, Grafana) 기반 메트릭·로그 통합 모니터링 구축
- ELK → EFK Stack 전환 후 ECK Operator 기반 Elastic Stack 9.0 CRD 선언형 관리로 재설계 (TLS 자동 회전, 롤링 업그레이드 자동화)
- SLI/SLO 기반 알림 체계로 MTTR(평균 장애 복구 시간) 단축 및 서비스 가용성 향상
- kube-prometheus-stack 기반 커스텀 알림 규칙·Grafana 대시보드·4종 DB exporter (MySQL, Redis, Elasticsearch, PostgreSQL) 통합

자동화 및 효율화

Python, Bash를 활용한 운영 자동화 및 개발 생산성 도구 구축

- 데이터 수집, 배포 프로세스, 문서 관리 자동화로 수동 작업 90% 이상 감소
- 내부 LLM 서비스(Ollama + Open WebUI) 구축으로 사내 보안 가이드라인 100% 준수 및 개발팀 생산성 향상
- GitLab Webhook 기반 문서 자동 동기화, Slack Bot 기반 APK 배포 알림 등 사내 도구 개발
- DP사 운영팀 Kubernetes 자체 운영 체계 확립 — 6개월 교육 프로그램 설계·운영 (수강자 평균 10명·회당 4시간), 교육 자료 표준화로 타 고객사 교육에도 재활용

Kubernetes 플랫폼 운영 고도화 및 오픈소스 기여

네트워크·로깅 스택의 무중단 전환과 재사용 가능한 공용 차트 오픈소스 공개

- Ingress-nginx 다수 인스턴스 → NGINX Gateway Fabric 단일 컨트롤플레인 + 클래스별 Gateway CR 무중단 마이그레이션 (전체 cutover, wildcard TLS 통합)
- ECK Operator 기반 로깅 스택을 로컬 차트 → OCI 차트로 무중단 전환 (CR/PVC 이름 보존, cluster_uuid 불변, 데이터 손실 없음)
- 오픈소스 Helm 차트 3종(nginx-gateway-cr, elasticsearch-eck, kibana-eck) 및 composite GitHub Actions 다수 공개, ArtifactHub / GitHub Marketplace 등록

데이터 엔지니어링 및 사내 LLM 서비스

GCP 데이터 파이프라인부터 사내 LLM 서비스까지 직접 구축·운영

- BigQuery + Dataflow + Cloud Functions + Cloud Scheduler 조합으로 멀티 데이터 소스(MongoDB·CloudSQL·GA·Dune) → BigQuery → Google Sheets 자동 파이프라인 구축, 인프라 100% Terraform IaC
- 온프레미스 GPU 서버 위 Ollama + Open WebUI 기반 사내 LLM 서비스 Kubernetes 배포·운영, 외부 AI SaaS 구독 대체로 월 약 \$100 절감 및 사내 보안 가이드라인 100% 준수
- 데이터 수집 자동화 95% 달성, 휴먼 에러 제거로 데이터 누락률 0% 달성

주요 경력 (Professional Experience)

팬텀(콘크리트 스튜디오)

2024.10 ~ 현재

DevOps Engineer

게임 개발사의 인프라를 단독으로 책임지는 DevOps 엔지니어로, AWS 기반 클라우드와 온프레미스 서버를 설계·구축·운영하고 있습니다. 개발 서버와 테스트 서버는 온프레미스에, QA와 Production 서버는 AWS에 구축하여 하이브리드 아키텍처를 운영 중이며, 이 구조로 월 인프라 비용을 20%(\$60,000 → \$48,000) 절감하고 절감분을 신규 프로젝트에 재투자하는 선순환을 만들었습니다.

현재 서비스 중인 라이브 게임에 대해 외부 퍼블리셔와 협업하여 2차 기술지원 및 운영을 담당하고 있으며, 개발 중인 신규 게임 프로젝트의 개발 생산성 향상을 위해 온프레미스·AWS 인프라를 구축하고 있습니다.

프로젝트 1: AWS-온프레미스 하이브리드 클라우드 아키텍처 구축

기여도 100% (단독 설계 및 구현, 외부 퍼블리셔 운영팀 협업) 2024.10 ~ 현재 (운영 중)

문제

전체 인프라를 AWS에서 운영하던 중 월 \$60,000의 높은 클라우드 비용이 발생하여 비용 최적화가 필요했습니다.

접근 전략

워크로드별 비용을 분석한 결과, 개발·테스트 환경은 트래픽 변동이 적어 클라우드의 자동 스케일링이 불필요했고, QA·Production 환경만 확장성과 안정성이 요구되는 상황이었습니다. 이에 개발·테스트 환경은 온프레미스로 전환하고, QA·Production 환경은 AWS에 유지하는 하이브리드 아키텍처를 설계했습니다.

양쪽 환경은 VPN 없이 역할을 분리해 독립 운영(앱 트래픽 분리)하며, 이미지 레지스트리도 환경별로 분리(개발=Harbor, QA·Production=ECR)해 아키텍처 차이에 맞춰 각 환경에서 독립적으로 빌드했습니다. 양쪽 환경 모두 Terraform과 Ansible을 활용한 IaC 통합으로 운영 표준을 일원화(신규 컴포넌트 IaC 100%, IAM 제외)하여, 운영 복잡도 증가 없이 일관된 인프라 관리 체계를 확립했습니다.

트러블슈팅

- EKS 환경에서 ALB Ingress 구성 시, 게임 클라이언트 버전별로 다른 백엔드로 라우팅해야 하는 요구사항 발생. ALB의 조건부 라우팅 규칙에서 커스텀 헤더 기반 라우팅과 Target Group을 조합하여 해결
- Kubernetes IPVS 모드에서 ClusterIP 서비스 간 통신 시 간헐적 타임아웃 발생. IPVS conntrack 임계치 튜닝으로 해결
- ArgoCD 로 관리하는 프로덕션 서비스들의 rolling deploy 중, ALB target deregistration delay(대상 등록 해제 대기 시간)와 Kubernetes Pod 의 SIGTERM 처리 시점이 어긋나면서 일부 세션 connection drop 발생. 다음 3가지 조치로 rolling deploy 중에도 connection drop 0건·zero-downtime 배포 달성:
 - (1) ALB drain 타이밍 정렬 — target group deregistration_delay 를 기본 300초에서 30초로 단축하고, Pod lifecycle.preStop hook 에 sleep 35s 추가. 종료 중 readiness probe 가 먼저 fail 하여 ALB 가 target 을 unhealthy 로 인식·신규 요청을 차단하되, in-flight 요청은 계속 처리해 무중단 확보. terminationGracePeriodSeconds 는 preStop(35s)을 초과하도록 설정 — 예를 들어 앱이 SIGTERM 시 in-flight 요청을 자체 drain(~30초)하는 서비스는 preStop 35s + drain 여유까지 합산해 75초, 그 외 서비스는 60초
 - (2) RollingUpdate 전략 조정 — maxSurge 는 절대값 1 로 고정(퍼센트 지정 시 노드가 다수 동시 생성되므로 절대값으로 제어)하고, maxUnavailable 은 replica 스케일에 맞춰 설정 — 예를 들어 replica 2~3이면 maxUnavailable 1로 surge-first(롤아웃 중 ~100% 용량 유지), replica 10이면 "50%"→0 반올림으로 surge-first zero-downtime, 대규모(replica 10~20)면 maxSurge 0 + maxUnavailable 20~25%로 신규 노드 생성 없이 배치 롤아웃
 - (3) 다중 replica(2 이상) 서비스에 PDB 적용 — Karpenter 노드 consolidation(노드 통합·축소) 시 Pod 을 하나씩만 축출하도록 보호 (단일 replica 서비스는 PDB 미적용)

성과

- 워크로드 분석 기반 인프라 최적화로 월 비용 20% 절감 (\$60,000 → \$48,000), 연간 약 \$144,000 절감분을 신규 프로젝트 인프라에 재투자하여 리소스 선순환 구조 구축
- 개발 서버 온프레미스 이전으로 월 약 \$1,000 AWS 리소스 비용 추가 절감 및 리소스 사용량 최적화
- 환경별 역할 분리로 운영 안정성 확보 및 장애 격리

기술 스택

AWS (EKS, EC2, ALB, Route53, ACM, EFS, Aurora MySQL, ElastiCache, CloudFront), Kubernetes, Terraform, Helm

프로젝트 2: 온프레미스 CI/CD 파이프라인 구축 및 고도화

기여도 100% (인프라 및 파이프라인 전체 담당) 2024.12 ~ 현재 (운영 중)

문제

개발자가 수동으로 빌드 후 서버에 배포하는 방식으로, 배포 시 평균 30분이 소요되었으며 휴먼 에러로 인한 장애가 빈번하게 발생했습니다.

접근 전략

GitLab CI와 ArgoCD를 조합한 GitOps 기반 자동 배포 환경을 구축했습니다. 개발자가 Git Push만 하면 자동으로 빌드→테스트→배포가 진행되며, ArgoCD가 Kubernetes 클러스터 상태를 모니터링하여 선언적 배포를 수행합니다.

Kaniko로 Docker-in-Docker(privileged) 없이 Docker 이미지를 빌드하고, 멀티 환경(alpha/review/dev/staging) 배포를 단일 파이프라인에서 관리하도록 구성했습니다.

트러블슈팅

- 환경 구분 없이 `selfHeal=true` 를 적용하면 prod 변경이 사람 승인 없이 자동 반영되고 긴급 대응 유연성도 떨어지는 문제 → prod 는 `selfHeal=false` + 수동 sync window 로 배포를 게이트하고 dev/staging 만 `selfHeal=true` 로 자동 drift 교정, 모든 sync 이벤트를 Slack 으로 알림 연동하여 변경 추적성과 안정성을 동시에 확보
- GitLab 업그레이드 후 GPG 키 만료로 CI Runner에서 패키지 설치 실패. apt-key 갱신 절차를 runbook으로 표준화·문서화하여 재발 시 신속 대응
- Kaniko 빌드 시 캐시 무효화로 빌드 시간이 급증하는 문제 발생. `-cache=true --cache-ttl=24h --snapshot-mode=redo` 옵션을 조합하여 캐시 적중률을 높이고 빌드 시간 변동을 최소화
- 데이터 배포 파이프라인의 소스 패치 병목을 점진적으로 튜닝 — 기존 `git checkout + git pull` 방식이 매 실행마다 전체 history(약 3.5GB)를 다시 fetch해 약 139초가 소요되던 것을, `git fetch --depth=1 --filter=blob:none` (최신 커밋만 + 파일 내용은 필요할 때만 받아 전송량 최소화)로 교체하여 소스 패치 약 139초 → 약 30초로 단축
- npm 의존성 설치를 `npm ci --cache .npm --prefer-offline` 에 lockfile 해시 기반 cache key를 결합해 tarball 다운로드를 재사용하도록 최적화(의존성 변경 시에만 캐시 무효화)
- 데이터 업로드 경로를 키리스로 전환 — 기존 SSH scp/rsync 직결 방식을, S3 transit 버킷에 스테이징한 뒤 SSM SendCommand로 대상 인스턴스가 `aws s3 sync --exact-timestamps` 를 수행하는 방식으로 교체하여 SSH 키를 완전히 제거하고 배포 계정 키 유출 리스크를 차단

성과

- 배포 시간 67% 단축 (30분 → 10분)
- 수동 작업 자동화 90% 달성 (빌드, 배포, 설정 적용)
- GitOps 자동 배포로 배포를 이원화 — dev/qa/review 는 머지 기반 지속 배포(핵심 서비스 repo당 주 100회 내외)로 신규 피쳐 Time to Market(시작 대응 속도) 단축, prod 는 수동 sync window 로 게이트한 주 5~7회 통제 릴리스로 안정성 확보, 배포 리소스 효율화로 개발팀이 기능 개발에 집중
- 크로스레포 MasterData 파이프라인 오케스트레이션 — 데이터 레포 변경 시 하위 서비스로 자동 fan-out(일부 서비스는 전 환경, 일부는 dev/qa 한정), 수신 측 파이프라인이 upstream 아티팩트를 받아 실제 변경 시에만 커밋 후 자체 빌드를 트리거하고, trigger 소스·타입으로 필터링해 순환 트리거(loop)를 원천 차단
- 기획팀·개발팀과 배포 정책을 함께 정의 — 데이터 업로드를 빌드 트리거로 삼고 환경별 배포 게이트(dev·qa·review 자동, prod 수동 승인)를 협의해 팀 간 배포 책임 경계를 명확화

기술 스택

GitLab CI/CD, ArgoCD, Jenkins, Kaniko, Kubernetes, Docker, Harbor

프로젝트 3: 중앙 로깅·관측 플랫폼 (로그 수집 → 무결성·HA → 제품 분석)

기여도 100% (로그 수집·무결성·제품 분석 전 주기 단독 설계·구축) 2024.11 ~ 현재 (누적 약 18개월)

로그가 분산된 온프레미스 환경에서 시작해 중앙 집중식 로깅을 3단계로 고도화(ELK → EFK → ECK Operator)하고, 수집기 데이터 무결성·HA를 강화한 뒤, 축적된 EFK 로그 파이프라인을 게임 유저 리텐션 분석 플랫폼으로 재활용했습니다. build → operate → improve → analytics 로 이어지는 관측 플랫폼 전 주기를 단독으로 설계·운영했습니다.

3-1. 중앙 집중식 로깅 시스템 구축 (ELK → EFK → ECK Operator 3단계 고도화, 2024.11 ~ 현재)

문제

온프레미스 환경에서 서버 및 컨테이너 로그가 분산되어 있어 장애 발생 시 원인 파악에 많은 시간이 소요되었으며, 통합 모니터링 체계도 없었습니다.

접근 전략

Phase 1 (ELK 구축): Elasticsearch, Logstash, Kibana, Filebeat를 활용한 중앙 집중식 로깅 시스템을 구축했습니다. 모든 서버와 컨테이너의 로그를 Filebeat로 수집하고, Logstash에서 JSON 파싱 및 필드 매핑을 처리한 뒤 Elasticsearch에 저장하여 Kibana 대시보드로 실시간 모니터링 및 검색이 가능하도록 했습니다.

Phase 2 (EFK 전환): Logstash JVM 메모리 부담(1GB+), Filebeat의 Helm 생태계 불일치, 로그 파이프라인 커스텀 확장성 한계를 해소하기 위해 Fluent Bit + Fluentd 기반 EFK Stack으로 전환했습니다. C 기반 경량 수집기인 Fluent Bit이 DaemonSet으로 각 노드에서 Kubernetes 메타데이터 자동 태깅과 함께 로그를 수집하고, Fluentd가 멀티 output을 지원하는 로그 가공/필터링을 처리한 뒤 Elasticsearch로 전달하는 구조로 리팩토링했습니다. 차트 업그레이드 자동화 스크립트를 개발하여 upstream Helm 차트 자동 업그레이드 시 커스텀 PVC 템플릿을 보존하도록 구성했습니다.

Phase 3 (ECK Operator 전환 + Stack 9.0 major upgrade): Elastic 공식 Helm Chart 업데이트가 2년 이상 중단되고 CRD 기반 선언형 관리(ECK)가 주류로 전환됨에 따라, Elastic Stack을 8.5.1에서 9.0.0으로 major upgrade하고 ECK Operator 3.3.2를 elastic-system 네임스페이스에 도입했습니다. Elasticsearch/Kibana를 Custom Resource로 재정의하고 로컬 차트(CR wrapper) 형태로 관리하여 repo convention과 정렬했으며, 병렬 배포 (monitoring → logging 네임스페이스) 후 cutover 전략을 채택하여 무중단 전환을 수행했습니다. ECK 기반 체계에서는 TLS 인증서 자동 발급/회전(1년 / 30일), elastic 계정 패스워드 Helm template 관리, CR spec.version 한 줄 변경으로 완료되는 롤링 업그레이드 체계를 확립하여 수동 운영 부담을 제거했습니다.

Phase 4 (AWS(EKS) 로깅 확장): 온프레미스 EFK 스택을 AWS(EKS)용 `-aws` 스택 5종(elasticsearch-aws / kibana-aws / fluent-bit-aws / fluentd-aws / eck-operator-aws)으로 template화하여 온프레미스와 동일한 로깅 체계를 AWS에 이식했습니다. 로그 수명주기는 ILM으로 hot → warm → cold

티어 이동 후 60일에 자동 삭제하고, S3 스냅샷은 SLM으로 90일 보관하도록 자동화했으며, IRSA(파드에 IAM 역할 부여) 기반 web-identity로 repository-s3 스냅샷 권한을 부여해 정적 키 없이 운영했습니다.

트러블슈팅

- 게임 서버 로그에 섞인 null 바이트(\x00)가 Logstash 파싱을 깨뜨리는 문제 → mutate 필터(gsub)로 사전 제거하는 전처리 단계를 추가해 해결
- 같은 필드명에 서로 다른 데이터 타입이 들어와 Elasticsearch 필드 매핑 충돌(mapper_parsing_exception) 발생 → 인덱스 템플릿에 타입을 명시하고 Logstash 타입 변환 필터를 적용해 해결
- 로그 양이 급증하면 Elasticsearch 디스크 워터마크를 넘겨 인덱스가 read-only로 바뀌는 문제 → ILM 정책으로 7일 지난 인덱스를 자동 삭제해 안정화
- [EFK] Fluent Bit이 NFS 볼륨에서 WAL 파일을 쓰지 못하는 문제 → upstream Helm 차트의 Pod 템플릿에 커스텀 PVC 볼륨 패치를 주입하고, 업그레이드 시 자동 보존되도록 스크립트에 반영해 해결
- [EFK] Fluentd output 버퍼가 넘쳐(buffer overflow) 로그가 유실되는 문제 → chunk_limit_size·flush_interval 튜닝과 overflow_action=block으로 해결
- [ECK] Kibana 9는 `server.ssl.enabled=true` 면 세션 쿠키에 Secure 플래그를 강제해 HTTP 환경에서 로그인에 막히는 문제 → `tls.disabled + publicBaseUrl http:// + secureCookies:false` 조합으로 HTTP 로그인 경로 복구
- [ECK] Fluentd 공식 이미지에 ES9 variant가 없어 ES8 variant를 써야 하는 상황 → `fluent-plugin-elasticsearch 5.4.4` 의 `suppress_type_name:true` 로 ES 9 호환성을 확보하고, NFS PVC에 active marker(경로 확인용 표식) 로그를 넣어 fluent-bit → fluentd → ES 전 경로를 End-to-End 검증
- [ECK] 3.3.x stackconfigpolicy-controller가 managedNamespaces 범위 밖 리소스를 17분마다 GET하면서 reconcile(상태 맞추기) error가 반복되는 문제 → managedNamespaces:[] (cluster-wide watch)로 우회, RBAC는 이미 cluster-scoped라 권한 변경 없음

성과

- 로그 검색 시간 90% 단축 (개별 Pod 접속 확인 → Kibana 단일 인터페이스), MTTR 개선으로 서비스 가용성 안정 유지
- EFK 전환으로 Logstash JVM(1GB+) 메모리 부담 해소 (Fluent Bit DaemonSet 수집 + Fluentd 가공 구조로 재설계), 일 평균 1GB 로그 실시간 처리 및 90일 보관
- ECK Operator 전환으로 TLS 인증서·계정 패스워드·StatefulSet 업그레이드의 수동 운영 부담 제거. CR spec.version 한 줄 변경만으로 롤링 업그레이드 가능
- Elastic Stack 8.5.1 → 9.0.0 major upgrade 완료 (2년간 누적된 업데이트 격차 해소)
- ECK Operator의 secureMode + ServiceMonitor로 controller-runtime 메트릭을 kube-prometheus-stack에 자동 등록, Operator 내부 상태까지 관측 가능한 체계 확립
- 온프레미스 EFK 스택을 AWS(EKS)용 `-aws` 스택 5종으로 template화해 동일 로깅 체계를 AWS에 이식, ILM hot/warm/cold 60일 삭제 + S3 스냅샷 SLM 90일 보관을 IRSA 기반으로 자동화

기술 스택

Elasticsearch, Fluent Bit, Fluentd, Logstash, Kibana, Filebeat, ECK Operator, CRD, Custom Resource, Kubernetes, Helm

3-2. Fluent Bit 로그 수집기 데이터 무결성 & HA 강화 (2026.03 ~ 2026.05)

문제

Phase 3 ECK 전환 후 운영 중 Fluent Bit Pod 재시작 시 tail input의 in-memory offset(어디까지 읽었는지 위치)이 사라지면서 동일 로그 재색인 / 로그 누락 위험이 발견되었습니다.

또한 단일 replica + PDB(Pod 중단 허용 정책)·preStop hook 부재로 노드 drain 시 짧은 수집 공백이 발생했고, 수집기 자체를 모니터링하는 알림 규칙이 없어 장애가 감지되지 않을 가능성이 있었습니다.

접근 전략

Tier 1 (데이터 무결성): tail input plugin(로그 파일 끝을 이어 읽는 입력)에 SQLite offset DB(`db /var/log/flb-storage/tail.db`)를 도입하고 전용 PVC(2Gi)를 마운트해 Pod 재시작 후에도 offset이 보존되도록 구성. helmfile revision 4에서 동일 로그 재색인 0, 중복 0 검증 완료.

Tier 2 (가용성): PodDisruptionBudget(`minAvailable=1`), preStop hook(`sleep + flush`), liveness/readiness probe(/api/v1/health) 추가로 노드 drain·롤링 업데이트 중 수집 공백 최소화.

Tier 3 (모니터링): PrometheusRule 7개 alert(input/output 실패율, retry 누적, fluentd queue 압력, fluent-bit Pod down, SQLite DB write fail 등) 추가로 감지되지 않던 장애를 사전 포착.

Tier 4 (HA 옵션 문서화): 향후 확장을 위한 3가지 옵션(입력 path 분할 / 사이드카 / Vector 멀티 수집기) 비교 문서화.

성과

- Pod 재시작 시 동일 로그 재색인 0건 · 로그 중복 0건 달성으로 로그 데이터 무결성 보장
- PDB + preStop hook 적용으로 노드 drain·롤링 업데이트 중 수집 공백 최소화, MTTR에 직접 기여
- 7개 PrometheusRule alert와 Slack alert routing으로 수집기 자체의 감지되지 않는 장애를 사전 차단하고 운영자 hand-off 경로를 확보, 오피버빌리티 stack 자기 모니터링 체계 확립
- HA 확장 옵션 3종으로 문서로 정리해 향후 트래픽 증가 시 의사결정 비용 절감

기술 스택

Fluent Bit (tail / SQLite offset), Kubernetes PVC / StorageClass, PodDisruptionBudget, Prometheus Operator (PrometheusRule), Helmfile, Fluentd (buffer tuning)

3-3. 게임 유저 리텐션 분석 플랫폼 (EFK 로그 → 제품 분석, 2026.06 ~ 현재)

문제

EFK 로깅 스택이 장애 대응·로그 검색 용도로만 활용되고 있었고, 게임의 신규 유저·리텐션·이탈을 정량적으로 파악할 제품 분석 지표가 없었습니다. 별도 분석 SaaS 도입 없이 기존 로그 파이프라인을 재활용해 리텐션 분석 체계를 만들 필요가 있었습니다.

접근 전략

기존 EFK 로깅 스택 위에 Elasticsearch Transform(5분마다 자동 실행)을 붙여, 이벤트 단위로 쌓인 원시 로그를 사용자 한 명당 한 줄로 요약한 인덱스(첫 접속일 first_seen / 활동일 active_dates / 최고 클리어 챕터 max_cleared_chapter)로 자동 집계했습니다. 날짜는 KST(한국 시간) 기준으로 맞추고, 이 인덱스 위에 Kibana 대시보드 12개 패널(신규/일간/주간/월간 활성 유저(NU/DAU/WAU/MAU), 신규·일간 유저 추세, D+1~D+30 잔존율(리텐션) 곡선, 일자별 코호트 잔존율 표, 챕터 분포)을 만들었습니다.

대시보드와 Transform 정의를 환경별로 그대로 찍어낼 수 있게 가이드를 문서화해 다른 환경에서도 재사용하도록 했습니다.

성과

- 장애 대응 중심의 로그 수집 스택을 제품 분석 플랫폼으로 확장 — 별도 분석 SaaS 없이 신규 유저·리텐션·챕터 진행을 정량 관측
- ES Transform 기반 per-user Cohort 인덱스 + 12패널 Kibana 대시보드로 D+1~D+30 리텐션 곡선·일별 Cohort 테이블 제공
- 대시보드·Transform 정의를 환경별로 템플릿화해 다른 환경에도 재사용 가능한 분석 자산으로 정리

기술 스택

Elasticsearch Transform, Kibana, EFK Stack, Runtime Field (KST), ECK Operator, Kubernetes

프로젝트 4: 개발 생산성 도구 구축 (APK 배포 봇 · 문서 자동화 · LLM 서비스 · Git 미러링 · 정적 파일 서버)

기여도 100% (전체 시스템 설계 및 개발) 2025.02 ~ 현재

개발팀의 반복적인 수동 작업을 자동화하고 생산성을 높이기 위해, 5가지 사내 도구를 설계·개발·운영했습니다.

4-1. Slack APK 배포 자동화 봇 (4주, 2025.02)

문제

APK 빌드 완료 후 QA팀 전달을 수동으로 하다 보니 공유 지연 및 버전 혼동 발생

해결

Jenkins APK 빌드 완료 시 Slack에 빌드 완료 메시지와 다운로드용 QR 코드를 자동 전송하는 Python 봇 개발, Kubernetes에 배포

성과

QA팀 전달 시간 90% 단축, QR 코드 기반 즉시 설치로 버전 혼동 제거, 빌드 이력 자동 기록

기술 스택

Python, Slack Bot API, Jenkins, Kubernetes, Docker

4-2. GitLab-Google Drive 문서 자동화 시스템 (2주, 2025.06)

문제

기술 문서를 GitLab에 마크다운으로 작성한 뒤 Google Drive에 수동 업로드하는 이중 작업이 발생

해결

GitLab Webhook과 Google Drive API를 연동하여 Push 시 자동 동기화. Python으로 마크다운→Google Docs 변환 후 업로드.

이후 업로드 잡 속도를 점진적으로 개선 — rclone `--checksum` 으로 체크섬을 비교해 미변경 파일 업로드를 스킵하고, sparse-clone으로 전체 repo가 아닌 대상 디렉토리만 클론하도록 최적화

성과

문서 관리 시간 80% 감소 (문서당 10분 → 2분), GitLab을 Single Source of Truth로 일원화

기술 스택

GitLab Webhook, Google Drive API, Python, Bash Script

4-3. 내부 LLM 서비스 구축 (4주, 2025.11)

문제

개발팀의 LLM 활용 수요 증가, 외부 SaaS 사용 시 비용 부담 및 사내 데이터 보안 우려

해결

온프레미스 GPU 서버에 Ollama와 Open WebUI를 Kubernetes 위에 배포. NFS 스토리지로 모델 저장소 구성

성과

개발팀 15명 대상으로 일 평균 100건 쿼리를 처리하여 코드 리뷰·문서 요약·디버깅 보조 도구로 활용. 사용 가이드 문서 작성과 개발팀 15명 온보딩 안내로 도구 정착을 직접 주도. 외부 AI SaaS 구독 대체로 월 약 \$100 절감. 사내 보안 가이드라인 100% 준수 및 외부 데이터 유출 리스크 제거

기술 스택

Ollama, Open WebUI, Kubernetes, NFS, GPU Server

4-4. Git 저장소 미러링 도구 개발 + Retry API 후속 강화 (4주 초기 개발, 2026.02 ~ 2026.06)

문제

AWS CodeCommit 은 응답이 느린 데다, CodeCommit·GitLab·GitHub 멀티 플랫폼 간 소스 코드를 수동으로 동기화하다 보니 누락과 지연이 발생했고, 내부 소스는 GitLab 으로 안정적으로 통합·이전할 필요가 있었습니다.

이후 운영 중 mirror 동기화가 실패하면 운영자가 `kubectll exec` 로 컨테이너에 진입해 webhook secret을 직접 노출하는 방식 외에는 수동 재시도 수단이 없어 SRE 친화적이지 않았고, AWS eu-central-1 transient blip 사고 이후 빠른 복구 수단 확보가 우선순위로 올라왔습니다.

해결

Phase 1 (초기 개발, 2026.02): Go로 양방향 Git 미러링 도구 git-bridge를 개발. CodeCommit은 SQS 폴링으로, GitLab·GitHub는 HTTP Webhook으로 변경 이벤트를 받아, 매번 전체가 아니라 바뀐 부분만 가져오는 증분 동기화(git fetch)와 무한 반복을 막는 루프 감지로 안정적으로 운영. 유닛 테스트와 CI/CD 파이프라인(개발은 GitLab CI, 공개 OSS는 GitHub Actions)으로 코드 품질을 확보하고 Kubernetes에 배포, Slack 알림을 연동.

Phase 2 (Retry API 후속 강화, 2026.05): 수동 재시도용 `POST /retry/mirror` 엔드포인트를 추가하고, 전용 토큰(`RETRY_API_TOKEN`, webhook secret과 분리)으로 인증 — 토큰 비교는 타이밍 공격을 막는 constant-time 방식. JSON으로 repo / direction / ref만 넘기면 API는 즉시 응답하고 실제 mirror는 백그라운드(goroutine)에서 재실행되며, 알림 출처를 `retry-api` 로 표시(EventMeta.Source)해 Slack에서 webhook 트리거와 구분. 방향은 `auto / source-to-target / target-to-source` 3종이며, repo별 `retry_direction` 우선값과 repo별 Slack webhook 환경 변수로 알림 채널까지 분리. 관련 테스트 11개 추가.

Phase 3 (안정성 가드, 2026.06): 양방향 미러에서 여러 곳의 조율되지 않은 force-push가 공유 브랜치를 옛 상태로 되돌리는 문제(비-fast-forward 회귀)의 근본 원인을 분석하고, ref 패턴별로 동기화 방향을 한쪽으로 고정하는 `ref_overrides` 를 설계. repo 전체는 양방향(bidirectional)으로 두되 특정 브랜치만 단방향으로 묶어, 반대 방향으로 되돌아오는 전파(역방향 echo)를 원천 차단. 밀어 넣는 push 범위도 방금 바뀐 ref로만 좁히고(전체 push인 `--all` 제거, 이 기능을 켜 repo에만 적용해 나머지 repo 동작은 그대로 유지), 브랜치 삭제(delete) 이벤트에도 같은 방향 제한을 적용. 가드가 실패하면 기존 동작(force)으로 되돌아가 정상 미러는 멈추지 않도록 하는 **fail-open(실패해도 막지 않고 통과)** 원칙으로 설계.

성과

- 저장소 10개에서 일 평균 30건의 동기화 이벤트를 처리하며 멀티 플랫폼 간 소스 동기화를 100% 자동화하고 수동 미러링 작업을 제거. 오픈소스로 공개해 GitHub Actions(multi-git-mirror)와 함께 활용
- Retry API 도입으로 webhook secret 노출 없이 안전한 수동 재시도 경로 확보. dev 환경에서 auto direction이 target-to-source로 해석되고 Slack TEST 채널 도착까지 검증 완료
- CI 테스트 게이트를 통과하고, AWS regional transient blip 같은 장애 상황에서 SRE 친화적인 수동 복구 수단을 확보
- 비-fast-forward 회귀를 막는 방향 고정 가드를 구현·검증. 테스트 전용 repo에서 회귀·신규·종합 런타임 검증 6/6 PASS, 프로덕션 repo는 격리해 영향 없이 유지. feat / fix / docs / chore 커밋 4개를 main에 반영

기술 스택

Go, AWS SQS, GitLab Webhook, GitHub Webhook, HTTP Bearer Auth, Slack Webhook, Kubernetes, Docker

4-5. 정적 파일 서버 자체 개발 (오픈소스, 2026.03 ~ 현재)

문제

기존에 사용하던 [halverneus/static-file-server](#)는 단순 파일 서빙만 제공하여 디렉토리 리스팅 UI, 파일 프리뷰, 검색/필터, 일괄 다운로드 등 실제 운영에서 필요한 기능이 없었습니다.

QA팀 APK 배포와 사내 파일 공유 환경의 사용성 개선 요구가 지속되었고, 관련 업스트림 차트의 유지보수도 사실상 멈춰 있었습니다.

해결

Go 기반 경량 정적 파일 서버(static-file-server)를 직접 설계·개발하고, GitHub Pages에 [Helm repository](#)를 운영하여 배포 표준을 확립했습니다. Kubernetes에는 자체 Helm chart로 배포하고 NFS 스토리지에 연결해 장기 보존 아티팩트를 안정적으로 호스팅하고, 기존 OSS 이미지(halverneus/static-file-server) 기반 배포를 완전히 대체했습니다.

주요 기능

- 다크모드 지원 디렉토리 리스팅 UI (그리드/리스트 뷰 전환, 키보드 네비게이션)
- 파일 프리뷰 (이미지 / 비디오 / 오디오 / PDF / 텍스트 · 코드)
- 검색 · 필터 + URL 해시 공유, 다중 선택 + ZIP 일괄 다운로드
- Gzip 압축, Prometheus 메트릭 노출, JSON 구조화 로깅, Health Check
- APK 파일 Content-Type 자동 설정 (ingress annotation 기반)

성과

Apache-2.0 라이선스로 오픈소스 공개(Docker Hub · GitHub Pages Helm repo 운영). 일 평균 30건 다운로드 및 주 5회(매일 빌드) APK 배포 트래픽을 처리하며 APK 배포·문서 공유·빌드 아티팩트 배포 등 사내의 여러 파일 공유 요구를 단일 도구로 수렴했고, 자체 Helm chart 기반으로 배포·업그레이드·롤백을 표준화했습니다.

업스트림 의존성 제거로 보안 패치 및 기능 추가를 자체 개발 주기로 관리할 수 있게 되었습니다.

기술 스택

Go, Helm Chart, GitHub Pages, Docker Hub, Kubernetes, NFS, Prometheus

프로젝트 5: Kubernetes 클러스터 운영 고도화

기여도 100% (단독 설계 — 모니터링·업그레이드 자동화·Helm 관리 프레임워크·NGF 마이그레이션·OSS Helm 차트 3종·스토리지 성능 최적화·Shell→Python 마이그레이션, 자동화 4 스크립트로 표준 절차 자산화 → 팀 재사용 가능) 2025.12 ~ 현재

클러스터 가시성, 운영 안정성, 네트워크 고도화, 스토리지 성능, 자동화 코드 품질을 목표로 모니터링 체계를 고도화하고 K8s 업그레이드·Helm 관리를 자동화했으며, 네트워크 컨트롤러를 Gateway API 기반으로 마이그레이션했습니다.

NGF 마이그레이션과 ECK Operator 도입의 산출물을 OSS Helm 차트로 공개하고, control-plane / DB 노드의 SSD 마이그레이션 및 빌드 I/O 격리로 스토리지 성능 병목을 해소했으며, 인프라 배포/업그레이드 파이프라인을 단계별(wave) 순차 자동화 + Shell → Python 마이그레이션으로 정비했습니다.

5-1. 모니터링 & 알림 고도화 (2025.12 ~ 현재)

문제

kube-prometheus-stack 기본 설치 상태로는 인프라·DB·네트워크 계층의 커스텀 알림·대시보드가 없었고, 물리서버와 VM 메트릭 수집 및 Cilium CNI 관측도 누락되어 장애 사전 감지가 어려웠습니다.

해결

kube-prometheus-stack을 기반으로 커스텀 알림 규칙과 Grafana 대시보드를 설계·제작했습니다. MySQL·Redis·Elasticsearch·PostgreSQL exporter를 통합하고, 물리서버와 VM에 Ansible로 node-exporter를 자동 배포했습니다.

Cilium CNI의 agent/operator 메트릭을 kubernetes_sd_configs(쿠버네티스 서비스 자동 탐색)로 수집하고, Alertmanager에서 severity 기반 Slack 라우팅과 inhibit rules(연관 알림 억제)를 구성해 알림 품질을 확보했습니다.

(메트릭을 사내에 보관하고 필요한 exporter 를 자유롭게 추가할 수 있다는 점, SaaS 구독료 부담이 없다는 점이 우선이라 Datadog / New Relic 같은 SaaS Observability 대신 kube-prometheus-stack 을 채택했습니다.)

성과

커스텀 알림 규칙과 대시보드로 인프라·DB·네트워크 전 계층의 장애를 사전에 감지하는 체계를 구축하고, 물리서버·VM까지 포괄하는 통합 관측 범위를 확보했습니다. severity 기반 라우팅과 inhibit rules로 중복 알림을 제거하여 온콜 대응 품질을 개선했습니다.

9개 컴포넌트 그룹(노드·Pod·Cilium·ArgoCD·Elasticsearch·Redis·MySQL·Harbor·control-plane)에 걸쳐 27개 알림 규칙을 운영하고 있습니다.

기술 스택

Prometheus, Grafana, Alertmanager, Cilium, node-exporter, Ansible, Slack API

5-2. K8s 업그레이드 자동화 & Helm 차트 관리 프레임워크 (2025.12 ~ 현재)

문제

Kubespray 기반 K8s 업그레이드가 수동으로 진행되어 사전 검증·백업·호환성 확인 단계가 일관되지 못했고, 인프라 전반 Helm 차트도 버전 관리 표준이 없어 업그레이드 스크립트가 차트별로 drift되었습니다. K8s 인증서도 1년 만료 주기로 API Server 장애 리스크가 있었습니다.

해결

Kubespray 버전 관리 자동화 스크립트(사전 검증·인벤토리 백업·버전 호환성 확인·breaking change(하위 호환 깨짐) 감지)와 업그레이드 후 다단계 헬스체크 스크립트(노드·Pod·DNS·인증서·etcd·API Server)를 구축했습니다.

인프라 전반 Helm 차트를 위한 업그레이드 스크립트 프레임워크를 구축하고, 4가지 캐노니컬 템플릿을 단일 source of truth로 관리하는 동기화 도구 (check 모드로 CI drift 감지, apply 모드로 일괄 전파)를 함께 개발했습니다.

K8s 인증서 자동 갱신 스크립트는 확인 → 백업 → 갱신 → kubelet·apiserver 재시작 → kubeconfig 업데이트 → 최종 검증의 단계별 플로우와 롤백을 지원하며, 노드 전반에 crontab으로 6개월마다 새벽 3시에 주기 실행됩니다.

(kOps는 AWS-only, Cluster API 는 온프레미스 bootstrap 이 무거워, Ansible 기반 자동화에 이미 익숙한 운영 환경에는 Kubespray 가 적합했기에 그대로 유지하며 자동화 레이어만 추가했습니다.)

성과

다단계 헬스체크 자동화로 K8s 업그레이드 후 검증 시간을 90% 단축하고, Helm 차트 업그레이드 프레임워크 + 캐노니컬 템플릿 동기화 도구로 인프라 버전 관리를 표준화하여 스크립트 본문 drift를 CI에서 자동 감지할 수 있게 되었습니다.

인증서 자동 갱신 스크립트와 crontab 주기 실행으로 노드 전반의 인증서 만료에 따른 API Server 장애를 사전에 제거했습니다.

기술 스택

Kubespray, Ansible, Helm, etcd, Shell Script, GitLab CI

5-3. Ingress-nginx → NGINX Gateway Fabric(NGF) 마이그레이션 (2026.04)

문제

온프레미스 클러스터에서 ingressClassName 기반 멀티테넌시를 위해 ingress-nginx 컨트롤러를 클래스별 다수 인스턴스(기본 + public-a~j, 클래스별 MetalLB LB IP 고정)로 운영하며 Deployment·Service·CRD·RBAC가 클래스마다 중복.

Kubernetes Gateway API가 v1.2 GA로 전환되며 ingress-nginx가 유지보수 모드로 들어갈 예정이라 Gateway API 기반 후속 컨트롤러로 이관이 필요했습니다.

해결

공식 후속 컨트롤러 NGINX Gateway Fabric(NGF) 2.x로 단일 컨트롤플레인 + 클래스별 Gateway CR 구조로 단순화. MetalLB IP를 그대로 유지하여 DNS·방화벽 변경 없이 병렬 배포 + 점진 cutover(트래픽 전환) 채택, Phase 0~7+ 전체(MetalLB 풀 확장 → NGF 설치 → 관측 스택 → HTTP-only/ApplicationSet → HTTPS 강제 앱 → IP swap cutover → Ingress 정리)를 **2일 내 완료**.

ingress-nginx 어노테이션을 Gateway API + NGF CRD(HTTPRoute filters / ClientSettings·ProxySettings·RateLimit·BackendTLSPolicy)로 1:1 매핑, HTTPRoute 차트 values 스키마를 여러 차트에 통일해 ApplicationSet 일괄 전환에 재사용. self-signed wildcard 인증서 1장(내부 도메인 전체, 10년)으로 앱별 TLS Secret을 통합.

트리블슈팅

- externalTrafficPolicy: Cluster + 고정 loadBalancerIP 상태에서 Pod 0 스케일로도 MetalLB가 IP를 해제하지 않아 NGF로 IP swap 불가.
Service를 ClusterIP로 patch하고 loadBalancerIP 를 제거하는 단계를 cutover 스크립트에 추가하여 강제 반환
- NGF chart 기본값 externalTrafficPolicy: Local (같은 노드 Pod로만 전달)로 인해 내부 Pod → 퍼블릭 도메인 → LB IP 호출 경로가 2분 timeout (dataplane 1 replica라 노드 local miss).
각 NginxProxy CR에 externalTrafficPolicy: Cluster 를 명시해 기존 ingress-nginx와 동일 정책으로 복원, CI generator 회귀 해결

성과

다수 컨트롤러 인스턴스 → 단일 NGF 컨트롤플레인 + 클래스별 Gateway CR로 CRD·RBAC·Deployment 중복 제거, 전체 클래스 cutover 사고 없이 완료 (클래스당 실 다운타임 30~60초).

HTTPRoute values 스키마 통일로 ApplicationSet 일괄 전환을 표준화, wildcard cert 통합으로 수동 갱신 부담 50% 감소, Phase 0~7+ 전 단계 2일 완료로 마이그레이션 기간 최소화.

기술 스택

NGINX Gateway Fabric, Gateway API, HTTPRoute, ClientSettingsPolicy, ProxySettingsPolicy, RateLimitPolicy, BackendTLSPolicy, Helmfile, MetalLB, ArgoCD

5-4. OSS Helm 차트 3종 공개 — nginx-gateway-cr · elasticsearch-eck · kibana-eck (2026.04 ~ 2026.05)

문제

NGF 마이그레이션과 ECK Operator 도입 과정에서 사내 클러스터 전용으로 만든 local CR chart 구조가 환경 의존(storageClass / CR 이름 / Secret 이름)이 많아 외부 재사용이 어려웠습니다. NGF upstream 차트는 컨트롤러·CRD만 설치하며 실제 트래픽 경로에 필요한 테넌트 레벨 리소스 (Gateway·NginxProxy·ReferenceGrant·ServiceMonitor·PodMonitor)를 제공하지 않았고, Elastic 공식 Helm Chart는 2년+ 업데이트가 정체된 상태였습니다.

사내 산출물을 공용 차트로 다듬어 공개하면서, 이미 로컬 차트로 운영 중인 기존 클러스터를 데이터 손실 없이 공개 차트로 전환할 경로도 함께 필요했습니다.

해결

nginx-gateway-cr: NGF 마이그레이션(5-3) 산출물에서 추출한 local CR chart 를 다듬어 Gateway / NginxProxy / ReferenceGrant / ServiceMonitor / PodMonitor 5종 테넌트 리소스를 배포하는 범용 차트로 공개했습니다. multi-Gateway 친화적 스키마(gateways[] 배열 + per-gateway override), NGF 2.x 권장 PodMonitor 템플릿(controller + dataplane), values.schema.json 입력 검증을 포함했습니다.

elasticsearch-eck / kibana-eck: elasticsearch-eck(11 ES 템플릿) + kibana-eck(10 Kibana 템플릿) 두 차트로

CR·Secret·HTTPRoute·Ingress·BackendTLSPolicy·ServiceMonitor·NetworkPolicy 를 단일 values 로 배포했습니다. values.schema.json (draft-07) 으로 HA / 스토리지 / 자원 입력을 검증하고 README 에 Minimal / HA 프리셋과 환경별 storageClass 매핑을 명시했습니다.

배포 채널 통일: 세 차트 모두 OCI(ghcr.io/somaz94/charts) + gh-pages Helm 레포 + ArtifactHub(networking / monitoring-logging 카테고리, Apache-2.0) 세 경로로 동시 배포했습니다. OCI 차트 버전 추적용 canonical 템플릿을 추가해 사내 버전 추적 절차도 정비했습니다.

무중단 전환: mgmt 클러스터를 로컬 차트 → OCI 차트(elasticsearch-eck / kibana-eck 0.1.1)로 전환할 때 CR·PVC·Secret 이름을 전부 보존하여 cluster_uuid 불변, ES pod 재시작 0회·Kibana rolling 1회로 데이터 손실 없이 전환을 완료했습니다.

성과

- 총 3개 차트(nginx-gateway-cr, elasticsearch-eck, kibana-eck)를 **Apache-2.0 라이선스**로 **ArtifactHub** 공개 — 사내 NGF + ECK 산출물을 외부 재사용 가능한 공용 자산으로 확보
- 사내 **helmfile 파이프라인** + 외부 **OSS 배포** 이중 활용 구조를 networking 스택과 logging 스택 양쪽에 확장
- HA 롤링 kind 검증 3/3 PASS, 실 클러스터 설치 Elasticsearch 71초 · Kibana 57초 만에 Ready/green 도달

- OCI 버전 추적 canonical 템플릿 신규 작성으로 향후 다른 OCI 차트의 버전 추적 절차도 동일 패턴 재사용 가능

기술 스택

Helm Chart, Gateway API, NGINX Gateway Fabric, ECK Operator, Elasticsearch, Kibana, Prometheus Operator, OCI Registry, ArtifactHub, Kubernetes

5-5. 스토리지 성능 최적화 — etcd / DB SSD 마이그레이션 + 빌드 I/O 격리 (2주, 2026.04 ~ 2026.05)

문제

온프레미스 클러스터의 control-plane VM과 게임 DB 호스팅 워커 노드(game-DB worker VM)가 HDD 위에서 동작하며 GitLab Runner buildx 작업과 디스크 I/O 경합 발생. etcd WAL fsync 지연 p99 약 2초 / apiserver pods GET p99 약 7초까지 치솟고, 게임 DB COMMIT latency가 9~14초로 폭증하여 실 플레이가 불가능해지는 장애가 반복되었습니다.

원인 분석 결과 GitLab Runner의 대량 write 트래픽이 game-DB worker VM 디스크의 fsync queue를 점유하는 것이 근본 원인으로 확인되었습니다.

접근 전략

1단계 (즉시 완화): MySQL을 `innodb_flush_log_at_trx_commit=2`, `innodb_flush_method=O_DIRECT_NO_FSYNC` 등 6개 파라미터로 튜닝해 fsync 압력 일시 완화.

2단계 (control-plane VM SSD migration): KVM/libvirt VM의 qcow2를 sparse copy(11GB, 6분 50초)로 SSD로 옮긴 후 virsh XML edit + cold start 절차로 다운타임 8.5분 만에 전환 완료(예상 15분 대비 단축). etcd 헬스체크 통과 후 Prometheus 지표로 즉시 검증.

3단계 (game-DB worker VM DB/Redis SSD migration): game-db (MySQL) + dev/qa/stg valkey-redis 총 35GB를 stop → backup → KVM SSD migration → restore → local-path PV 재바인딩 순서로 데이터 손실 없이 이전.

4단계 (build-only VM 신설): 별도 물리 서버에 4 vCPU / 16GB RAM / 150GB SSD VM을 cloud-init으로 자동 프로비저닝, kubespray로 클러스터에 join하고 `node-role=build` label + `dedicated=build:NoSchedule` taint를 부여. GitLab Runner Helm values에 nodeAffinity + toleration을 추가해 buildx 작업을 build-only 노드로 격리.

4종 자동화 스크립트와 설정 파일을 함께 정리해 향후 추가 워커 노드도 동일 절차로 생성 가능.

트러블슈팅

- YAML이 콜론이 들어간 MAC 주소를 60진수 숫자로 잘못 읽어(`0a:bb:cc:dd:ee:0a` 에서 앞 `0a` 가 사라짐) cloud-init VM이 부팅 후 네트워크 카드를 못 잡던 문제. cloud-init network-config의 MAC 값을 따옴표로 감싼 문자열로 명시하고, VM 검증 스크립트에 MAC 일치 확인 단계를 넣어 재발 방지
- control-plane VM을 완전히 켜다 켜(cold start) 직후 etcd가 잠깐 새 리더를 뽑는 과정(leader election)에 들어가 apiserver가 약 30초간 5xx 에러를 반환.
미리 kube-apiserver의 readiness(준비 완료) 판정 기준을 완화하고, 마이그레이션 절차에 'etcd 정족수(quorum) 회복 확인' 단계를 명시적으로 추가

포스트모템 · 재발 방지

- MAC 60진수 파싱 함정 → MAC 값을 따옴표 문자열로 명시 + VM 검증 스크립트에 MAC 일치 확인 단계 추가를, 다음에도 자동적으로 걸러지도록 runbook 절차(게이트)로 자산화
- control-plane VM cold start(완전 재부팅) 시 etcd 리더 재선출로 생기던 5xx → 'etcd 정족수(quorum) 회복 확인' 단계를 마이그레이션 절차에 명시적으로 추가해, 다음에 VM SSD 이전을 다시 하더라도 같은 5xx가 나오지 않도록 차단
- 자동화 스크립트 4종과 설정 파일을 표준 절차로 정리 — 다음에 워커 노드를 추가할 때 같은 검증 절차(11개 항목 PASS)를 그대로 재사용 가능

성과

- etcd WAL fsync 지연 1760ms → 3.88ms (454배 개선), etcd backend commit 1810ms → 3.77ms (480배 개선), apiserver pods GET p99 6520ms → 23ms (283배 개선) — Kubernetes API 가용성 회복
- 게임 DB COMMIT 지연 9~14초 → ms 수준으로 회복으로 GitLab Runner 빌드 작업 중 게임 플레이 불가 장애 0건 달성
- build-only VM 신설 검증 11 PASS / 0 WARN / 0 FAIL, 빌드 워크로드와 DB 워크로드의 물리적 격리 달성
- 자동화 4 스크립트 + 설정 파일 자산화로 향후 워커 노드 추가 시 동일 표준 절차 재사용 가능 (운영 표준화)

기술 스택

KVM / libvirt, qcow2, virsh, etcd, Prometheus / PromQL, MySQL, Redis (valkey), kubespray, cloud-init, GitLab Runner

5-6. 인프라 배포/업그레이드 자동화 + Shell → Python 마이그레이션 (2026.03 ~ 현재)

문제

사내 Kubernetes 인프라 monorepo(여러 컴포넌트를 한 저장소로 관리) 의 25개 컴포넌트가 helmfile + shell script 약 39,000줄로 관리되어 신규 컴포넌트 배포 / 차트 업그레이드 / canonical template(표준 원본 템플릿) 동기화가 전부 수동이었습니다. shell + awk 조합은 멀티 환경 분기·복잡한 YAML 패칭·테스트 가능성에서 한계를 드러냈고, CI 환경에서 helmfile + plugin 설치만으로 5분이 소요되어 매 잡마다 누적 비용이 컸습니다.

접근 전략

배포 자동화: CI 파이프라인을 6 단계 wave(bootstrap → core → security → db-redis → observability → apps)로 구성하고, MR diff 감지 → 변경 컴포넌트만 helmfile diff/apply, manual gate로 prod-safe 단계별(wave) 순차 배포를 표준화. CI script 10개(helmfile-changed-dirs, diff-component, apply-component, auto-upgrade, render-mr-body 등)를 새로 정리.

업그레이드 자동화: CI가 25개 컴포넌트 차트의 신버전을 감지해 자동 MR을 생성하고, 본문에 변경 chart·values diff·breaking change 노트를 자동 렌더링.

Shell → Python 마이그레이션: stdlib(pathlib, subprocess, re, json, argparse)만으로 8 종 canonical template(external-standard / external-oci / local-with-templates / cr-version 등)을 모듈화하고, 동기화·백업 스크립트 + 6개 CI script(총 1,387줄)도 Python으로 재작성. 25개 컴포넌트의 업그레이드 모듈과 758개 unittest를 함께 정비.

helmfile-tools 이미지 활용: 위 프로젝트 7에서 도입한 공용 Toolchain Image를 적용해 CI install 시간 5분 → 약 10초로 단축. 로컬(macOS venv) + CI 컨테이너 양쪽 모두에서 동일 코드가 동작.

성과

- 25개 컴포넌트 단계별(wave) 순차 자동 배포 + 업그레이드 MR 자동 생성 구축으로 인프라 배포/업그레이드 운영 부담 대폭 절감
- Shell → Python 마이그레이션 (사내 Kubernetes 인프라 monorepo 의 helmfile + shell 약 39,000줄 + 외부 자동화 shell 포함 합계 약 50,000줄, 758 unittest 통과), 멀티 환경 분기·복잡 YAML 패칭의 가독성·테스트 가능성 동시 확보
- helmfile-tools image 적용으로 CI 환경 준비 시간 5분 → 약 10초 (약 95% 단축), 잡 단위 누적 비용 제거
- 로컬(macOS venv) + 원격(CI container) 동일 코드 보장으로 개발 → 배포 사이의 환경 차이 이슈 제거
- 이 helmfile push 배포 모델은 이후 5·7의 ArgoCD pull-GitOps 컨트롤플레인으로 전환되어, push 기반 배포를 pull 기반 선언 관리로 대체 (배포 모델 진화)

기술 스택

Python 3.10+ (stdlib only), unittest, GitLab CI/CD, Helm / Helmfile, Git automation, yq

5-7. helmfile Push → ArgoCD Pull-GitOps 전환 (App-of-apps 컨트롤플레인, 2026.06 ~ 현재)

문제

플랫폼 릴리스를 helmfile push 방식으로 배포하다 보니 배포 주체가 CI 러너에 묶여 있었고, 멀티클러스터로 확장하면서 클러스터별 상태 드리프트를 선언적으로 수렴시킬 pull 기반 GitOps 컨트롤플레인이 필요했습니다.

접근 전략

helmfile push 배포(CI 러너가 클러스터로 직접 밀어 넣는 방식)를 ArgoCD pull-GitOps(클러스터가 Git의 목표 상태를 스스로 당겨와 맞추는 방식)로 전환했습니다. Matrix + Git files 제너레이터 기반 ApplicationSet(여러 Application을 자동으로 찍어내는 컨트롤러)이 각 컴포넌트의 `argocd/*.yaml` 마커 파일을 읽어 Application을 자동 생성하도록 구성해, 온프레미스 16개 + AWS 8개 총 24개 플랫폼 릴리스를 2개 클러스터에 등록했습니다.

무중단 adoption(관리 인수): 기존 helm 릴리스를 지우고 다시 만들지 않고 그대로 ArgoCD 관리로 넘겨받고(adopt,

`resourceTrackingMethod=annotation`), helmfile의 배포 순서(deploy-wave)를 ArgoCD sync-wave에 1:1로 매핑해 순서를 보존했습니다.

App-of-apps 부트스트랩: 상위 Application이 하위 Application들을 만드는 계층 구조(App-of-apps)로 root → {app,infra}-projects(wave -1) → {app,infra}-appsets(wave 0) 순서로 컨트롤플레인을 세우고, 연쇄 삭제 방지(prune-cascade) 안전장치로 컨트롤플레인 레이어는 `prune:false + selfHeal:true`, 말단(leaf) Application만 `prune:true` 로 분리해 실수로 인한 대량 삭제를 차단했습니다. 업그레이드는 버전 고정(argocd-pin) 패턴으로 관리했습니다.

ArgoCD resource.exclusions(추적 제외): 컨트롤플레인이 대량으로 만들어내는 리소스(`Endpoints` / `EndpointSlice` /

`coordination.k8s.io Lease` / 인증·인가(Authz·Authn) 리소스)를 ArgoCD 추적 대상에서 빼서, 감시 이벤트와 화면 잡음(UI clutter)을 줄이고 컨트롤러 부하를 완화했습니다(온프레미스·AWS 양쪽 argo-cd에 동일 적용).

AppProject 최소권한 whitelist(허용 목록): 테넌트(입주 서비스) 프로젝트의 `clusterResourceWhitelist` / `namespaceResourceWhitelist` 를 * (전체 허용) 대신 차트가 실제로 만드는 리소스 종류(kind)만 명시했습니다. 클러스터 전역 리소스는 `Namespace` · `PersistentVolume` 만 허용하고 `ClusterRole` / `CRD` 등은 일부러 안 줘서 테넌트가 클러스터 리소스를 못 만들게 했으며, infra 프로젝트만 `CRD`/`ClusterRole`/`Webhook` 설치를 위해 `*/*` 로 분리했습니다. 허용 목록에 없는 종류는 sync가 실패해 초기에 걸러집니다.

성과

- helmfile push → ArgoCD pull-GitOps 전환으로 2개 클러스터 24개 플랫폼 릴리스(온프레미스 16 + AWS 8)를 단일 컨트롤플레인에서 선언 관리
- 기존 helm 릴리스를 재생성 없이 adopt(재생성 없이 관리 인수, annotation tracking)해 무중단 전환 달성, deploy-wave → sync-wave 매핑으로 배포 순서 보존
- app-of-apps 연쇄 삭제 방지(prune-cascade) 안전장치(컨트롤플레인 `prune:false` / leaf `prune:true`)로 GitOps 대량 삭제 리스크 차단
- ArgoCD resource.exclusions로 컨트롤플레인이 대량 생성하는 리소스(`Endpoints`/`EndpointSlice`/`coordination.k8s.io Lease`/`Authz·Authn`)를 추적 제외 — 감시 이벤트·UI 잡음(clutter)·컨트롤러 부하 완화 (온프레미스·AWS 양쪽 적용)
- AppProject whitelist를 * 대신 차트 렌더 kind만 명시(클러스터 스코프는 `Namespace·PersistentVolume`만, infra만 `*/*`)해 최소 권한·장애 영향 범위(blast-radius) 축소, 허용 목록에 없는 종류는 sync 실패로 조기 검출

기술 스택

ArgoCD, ApplicationSet (Matrix / Git files generator), GitOps, Helm / Helmfile, Multi-cluster, Kubernetes

프로젝트 6: 인프라 보안 체계 구축

기여도 100% (설계, 배포, SSO 연동) 2026.04 ~ 현재

인프라 운영에 필요한 민감 정보를 안전하게 관리하기 위한 보안 체계를 구축하고 있습니다.

6-1. Vaultwarden 비밀번호 관리 시스템 (1주, 2026.04 ~ 현재)

문제

사내 시크릿·계정 자산이 Slack / email / 개인 vault 에 분산 보관되어 노출·중복 관리 리스크가 누적. 외부 SaaS 비밀번호 매니저 사용 시 운영 비용 + 사내 데이터 외부 저장 우려.

해결

Vaultwarden(Bitwarden 호환 서버)을 Kubernetes 에 Helm 차트로 배포하고, GitLab SSO(OpenID Connect)를 연동해 기존 계정 그대로 로그인하도록 구성했습니다. Self-signed TLS 인증서를 extraObjects 로 Helm 에서 관리하고, 조직/컬렉션 기반 팀별 접근 제어 (RBAC) 를 설정했습니다.

자동 백업 CronJob (날짜별 SQLite 덤프, 30일 retention) + 대화형 restore 스크립트로 데이터 안정성을 확보했습니다.

성과

- 사용자 70명 · 시크릿 50건을 중앙 관리 체계로 전환하여 분산 보관 리스크 제거
- 외부 SaaS 구독 비용 0원 유지, 사내 데이터 외부 저장 회피
- 백업/restore 절차 자동화로 사내 시크릿 자산의 운영 부담 최소화

6-2. Keycloak Operator 기반 중앙 인증(SSO) 시스템 도입 (2026.04 ~ 현재)

문제

ArgoCD·Harbor·Vaultwarden 등 사내 인프라 도구가 각각 GitLab을 OIDC IdP(로그인 인증을 제공하는 신원 제공자)로 직접 연동하다 보니, GitLab 자체의 사용자·세션·MFA(다중 인증) 정책에 의존하는 구조가 쌓였습니다.

앞으로 LDAP·MFA·세션 정책을 한곳에서 관리하려면, 로그인을 중개하는 별도의 중앙 인증 서버(broker, Keycloak)가 필요한 시점이었습니다.

접근 전략

Keycloak을 선언형으로 관리하는 Keycloak Operator + KeycloakCR + KeycloakRealmImport를 PostgreSQL 백엔드와 함께 도입하고, Keycloak이 GitLab 로그인을 중개(brokering)하도록 구성해 사용자는 기존 GitLab 계정 그대로 로그인합니다. Realm / Client / IdP / Group / Mapper 설정은 `kcadm.sh` 부트스트랩 스크립트로 코드화(codify)하고, Harbor·ArgoCD·Vaultwarden이 GitLab에 직접 연동하던 OIDC를 Keycloak을 거치도록 전환했습니다.

권한 관리(RBAC)는 GitLab group(`server` , `global-admin`)을 Keycloak group mapper로 중개한 뒤 ArgoCD의 role mapping에 그대로 연결해, 권한 운영의 단일 기준을 Keycloak으로 통일했습니다.

트러블슈팅

- IdP의 `providerId` 를 `oidc` 로 잘못 지정해 GitLab 로그인 중개(brokering)가 작동하지 않던 문제 — `providerId: gitlab` (커스텀)으로 수정
- GitLab IdP의 `defaultScope` 에 `openid` 만 있어 사용자 그룹 정보(group claim)가 빠지던 문제 → `read_user` 와 group 관련 scope(요청 권한 범위)을 추가 매핑해 ArgoCD 권한(RBAC)이 정상 동작
- Group mapper의 `claim.value` 와 KeycloakRealmImport YAML 스키마가 맞지 않아 mapper 설정이 적용되지 않던 문제 — `config:` 하위 key 형식을 Keycloak Operator 스펙에 맞게 수정
- Operator가 Realm 변경을 원하는 상태로 계속 맞추는(reconcile) 과정에서, 같은 작업을 반복해도 결과가 같아야 하는(idempotent) 성질이 없는 일부 필드(`client.secret` 등)가 무한 재생성되던 문제 → 해당 필드를 Operator 밖에서 Secret으로 따로 관리하도록 분리
- Harbor의 OIDC 전환(cutover) 때 기존 사용자 매핑이 어긋나 일부 계정의 권한이 사라질 위험을 발견 → Harbor의 `oidc_user_id` 매핑을 미리 뽑아 검증한 뒤 전환해 다운타임·권한 손실 0건 달성

성과

- ArgoCD / Harbor / Vaultwarden OIDC 통합 완료, 무중단, 검증 PASS — GitLab 직접 연동 의존성 해소
- GitLab group(`server` , `global-admin`) → Keycloak group mapper → ArgoCD role 의 단일 권한 흐름 확립, 권한 운영의 단일 기준을 Keycloak으로 일원화
- Keycloak Operator 도입 과정에서 발견된 5가지 함정(IdP providerId / scope / mapper config / Operator idempotency / Harbor user mapping)을 plan 문서에 정리해 향후 클러스터 재구축 시 재발 방지
- 향후 LDAP 연동(federation), MFA 강제, 세션 정책 중앙화의 기반 마련 (LDAP federation은 후속 트리거 대기)

기술 스택

Keycloak Operator, KeycloakCR / KeycloakRealmImport, PostgreSQL, OIDC / OAuth2, kcadm.sh, ArgoCD, Harbor, Kubernetes Gateway API

6-3. GitOps 시크릿 관리 — External Secrets Operator · Sealed Secrets (2026.05 ~ 현재)

문제

DB 비밀번호 등 민감 값이 Helm values에 평문으로 남아 Git에 그대로 커밋되는 구조였고, Harbor image-pull secret을 네임스페이스마다 수동으로 복제하고 있어 시크릿이 흘러지고 노출 리스크가 있었습니다.

해결

External Secrets Operator를 도입해 ClusterSecretStore ↔ AWS Secrets Manager를 연결하고, DB 비밀번호 평문 values를 ExternalSecret 참조로 전환했습니다. ExternalSecret이 Deployment보다 1 sync-wave(ArgoCD 배포 순서 단계) 먼저 동기화되도록 wave를 지정해 시크릿 부재로 인한 기동 실패를 방지했습니다.

Harbor image-pull secret은 cluster-wide SealedSecret으로 암호화해 admin-only AppProject(ArgoCD 접근 경계)에서 관리하고 각 네임스페이스에 안전하게 전달했으며, ArgoCD 알림은 notify-channel 라벨 라우팅으로 Slack 채널을 분리했습니다.

성과

- DB 비밀번호 등 민감 값을 Git 평문에서 제거하고 AWS Secrets Manager를 단일 출처로 일원화
- Harbor pull secret을 cluster-wide SealedSecret으로 표준화해 네임스페이스별 수동 복제 제거

기술 스택

External Secrets Operator, AWS Secrets Manager, Sealed Secrets, ArgoCD, Helm, Kubernetes

성과

- 사용자 70명 · 시크릿 50건을 중앙 관리 체계로 전환하여 보안 리스크 해소 및 인수인계 효율화
- GitLab SSO 연동으로 기존 사내 계정으로 즉시 로그인 가능, 별도 계정 발급/삭제 관리가 불필요해져 계정 관리 효율성 향상
- Keycloak 기반 중앙 인증 도입 — Harbor·ArgoCD·Vaultwarden 3개 시스템 OIDC 연동, GitLab IdP 직접 의존 제거, 5가지 함정을 plan 문서로 자산화

기술 스택

Vaultwarden, Helm, OpenID Connect, GitLab SSO, Kubernetes

프로젝트 7: 공용 Toolchain Image + 자동 버전 사이클

기여도 100% (아키텍처 설계 · CI 표준화 · 자동화 파이프라인 개발) 2025.09 ~ 2026.05

문제

GitLab CI 표준화 범위 안의 15개 repo(1개 template provider + 14개 consumer)에서 각 잡이 alpine 베이스에 helm-helmfile-kubectl-crane-aws-cli 등을 매번 설치하면서 첫 번째 잡 준비 시간이 평균 5분에 달했고, 도구 버전이 14개 consumer repo에 흩어져 있어 신버전 추가 시 일괄 갱신이 사실상 불가능했습니다. `:latest` 태그 의존으로 재현성이 떨어지고 취약점 추적도 어려웠습니다.

접근 전략

Layer 1 (Mirror): 외부 레지스트리(Docker Hub, GHCR)의 도구 이미지를 `crane copy` 로 사내 Harbor에 multi-arch manifest(여러 CPU 아키텍처를 묶은 이미지 목록) 보존 미러링.

Layer 2 (Custom): Layer 1 위에 도구가 사전 설치된 용량이 큰 이미지 7종(helmfile-tools, node-build, go-build, rclone-backup, aws-cli-tools, kaniko-build, terraform-tools)을 빌드해 consumer repo가 image pin만으로 도구를 즉시 사용할 수 있도록 표준화.

Tier 1~2 SSOT(단일 관리 기준): `.toolchain_build_custom` 공용 template + `TOOLCHAIN_*_TAG` 중앙 변수를 도입해 consumer repo는 image tag를 직접 추적하지 않도록 간접화.

공용 CI 템플릿 SSOT: repo마다 복불하던 CI 잡을 중앙 공용 CI 템플릿 repo로 추출해 각 repo가 `include` 로 참조하도록 전환. auth(키리스 AWS 인증) / build(kaniko) / deploy(ArgoCD) / backup(Google Drive) 잡 템플릿과 공용 앵커(git-ssh-rules-job-config-packages)를 단일 소스로 관리해 복불 드리프트를 제거.

Phase 4 자동 사이클: cron 트리거로 다음 4단계가 자동 진행되어 사람의 수동 click 2회(pipeline trigger + Tier 2 MR merge)만으로 연쇄 전파 완성:

- (1) upstream 신버전 감지
- (2) Layer 1 mirror auto-MR
- (3) Layer 2 rebuild + Tier 2 TOOLCHAIN_*_TAG 동기화 auto-MR
- (4) 14개 consumer repo(template provider repo 제외)에 자동 전파

성과

- CI 첫 번째 잡 준비 시간 5분 → 약 10초 (약 95% 단축), 14개 consumer repo에 동일 표준 적용으로 도구 버전 일관성 100% 확보 (template provider 포함 총 15개 repo)
- 버전 거버넌스 4-tier 구조(ARG → 중앙 변수 → consumer 간접화 → lint drift 감지)로 흩어진 12+ 버전 포인트 압축
- Phase 4 cron 자동 사이클로 helm-helmfile-kubectl-crane-rclone 등 5+ 도구의 신버전 연쇄 전파를 사람 클릭 2회로 완료, 매주 신버전 추적이 운영 부담 없이 가능
- minor-guard 정책(helm/helmfile/kubectl)으로 호환성 깨짐 사전 차단 (예: helmfile 1.0.0 → 1.5.1 연쇄 전파 후 helm >= 3.18.6 호환성 검증)
- 25개 컴포넌트를 리스크 티어(T1 stateless 주간 / T2 platform 격주 / T3 stateful 월간 + pre-flight)로 분류한 자동 업그레이드 파이프라인 구축 — 신버전 감지 → 컴포넌트별 자동 MR → Slack 알림 → wave 순차 적용, MAJOR_PIN 가드와 T3 atomic 롤백으로 안전성 확보
- docker:dind 사이드카(컨테이너 안에서 Docker를 실행하는 보조 컨테이너) + docker buildx(docker-container driver)로 amd64 + arm64 멀티아치 매니페스트 리스트를 빌드하고, 외부 이미지(alpine-node-golang-rclone-awscli-docker)를 crane으로 Harbor에 미러링해 외부 레지스트리 의존을 제거

기술 스택

GitLab CI/CD, Docker Buildx (multi-arch), Docker BuildKit (docker-container driver), crane, Kaniko, Harbor, Helm / Helmfile, Bash / yq, Slack Webhook

프로젝트 8: AWS EKS 프로덕션 게임 런칭 플랫폼 구축 (신규 외부 퍼블리싱 게임)

기여도 100% (그린필드 EKS 단독 설계·구축) 2026.06 ~ 현재 (구축·운영 중)

문제

라이브 게임 estate(AWS QA·Prod)와 별개로, 신규 외부 퍼블리싱 게임은 온프레미스 개발 클러스터 단일 체계로만 운영 중이었고, 프로덕션 런칭을 위해 확장성·가용성을 갖춘 별도 AWS EKS 프로덕션 환경이 필요했습니다. 온프레미스만으로는 글로벌 트래픽 대응과 오토스케일링에 한계가 있었습니다.

접근 전략

eu-central-1 리전에 그린필드 EKS 클러스터를 구축해 온프레미스 개발 + AWS 프로덕션의 멀티클러스터 하이브리드로 확장하고, 플랫폼 컴포넌트 약 11종을 전부 GitOps(ArgoCD)로 선언 관리했습니다.

오토스케일링·네트워킹: Cluster Autoscaler 대신 Karpenter를 채택(여러 스팟 family 통합·빠른 provisioning)해 9개 스팟 인스턴스 family에 걸친 스팟 오토스케일링과 게임 서버 전용 on-demand NodePool을 구성하고, AWS Load Balancer Controller(ALB/NLB) + ExternalDNS(Route53) + wildcard ACM으로 인그레스·DNS·TLS를 자동화했습니다.

로깅·시크릿: eck-operator 기반 EFK 스택(Elasticsearch 3노드 3-AZ, EBS gp3, S3 스냅샷 SLM)을 HA로 구성하고, External Secrets Operator로 시크릿을 외부화했습니다. 모든 ServiceAccount 인증은 IRSA / Pod Identity로 처리해 정적 키를 제거했습니다.

인증·배포: ArgoCD에 GitLab OAuth SSO + 게임 스크립트 RBAC를 연동하고, 클라이언트 아티팩트는 Jenkins → S3 + CloudFront로, 서버 매니페스트는 S3 transit 버킷 + SSM SendCommand(SSH 없이 원격 명령 실행)로 bastion의 EFS 마운트에 전달했습니다. bastion을 Name 태그로 동적 탐색해 SSH 키 없이 배포했습니다.

트리블슈팅

- 게임 프로덕션 Pod가 실제 사용량 대비 과도한 메모리(2Gi)를 예약해 노드가 불필요하게 증설되는 node sprawl 발생 → 실측 기반으로 memory requests를 512Mi로 조정해 노드 증설을 정상화하고 스팟 비용 효율 확보
- 롤링 배포 중 ALB가 아직 준비되지 않은 Pod로 트래픽을 전달하는 문제 → ALB Pod readiness gate + PodDisruptionBudget + graceful drain(연결을 안전하게 종료)을 조합해 무중단 배포 확보

성과

- 온프레미스 개발 단일 체계 → 온프레미스 + AWS 프로덕션 멀티클러스터 하이브리드로 확장, 신규 외부 퍼블리싱 게임의 클라우드 프로덕션 런칭 기반 확보
- 약 11종 플랫폼 컴포넌트를 100% GitOps(ArgoCD)로 선언 관리, IRSA / Pod Identity로 정적 키 없는 인증 체계 확립
- SSM SendCommand 기반 키리스 배포로 bastion SSH 키 관리 부담과 키 유출 리스크 제거
- ArgoCD 앱 마커와 프로덕션 CI/CD 전환 문서로, 단독 구축이지만 팀이 동일 절차로 운영·재구축 가능하도록 인수인계 경로 확보

기술 스택

AWS EKS (eu-central-1), Karpenter, AWS Load Balancer Controller, ExternalDNS, External Secrets Operator, ECK Operator / EFK, ArgoCD, IRSA / Pod Identity, AWS SSM (SendCommand), S3 / CloudFront, Kubernetes

너디스타

2023.03 ~ 2024.07

DevOps Engineer

게임·블록체인 기반 스타트업에서 DevOps 엔지니어로 근무하며, AWS에서 GCP로 대규모 클라우드 마이그레이션을 총괄했습니다.

GCP Startup Program 기반 비용 최적화와 글로벌 네트워크 성능을 목표로, 게임 점검 시간을 활용해 최소 다운타임으로 전체 서비스를 이전했습니다. 소스 저장소는 온프레미스를 GitLab, Production을 GitHub로 이원화해 관리했습니다.

프로젝트 1: AWS → GCP 대규모 클라우드 마이그레이션 및 CI/CD 구축

기여도: 인프라 아키텍처 설계·마이그레이션 전략 수립 및 실행 총괄 70% (나머지 30% 는 서버 개발팀의 application 측 마이그레이션 협업), CI/CD 파이프라인 구축 100% 10개월 (2023.03 ~ 2023.12)

문제

AWS 비용이 지속적으로 상승하고 있었으며(월 \$10,000+), 특히 NAT Gateway와 데이터 전송 비용이 높았습니다. 마침 GCP Startup Program 참여 기회가 생겨, 비용 절감과 멀티 리전 게임 서비스를 위한 글로벌 로드밸런싱 확보를 목표로 GCP 마이그레이션을 추진했습니다.

접근 전략

3단계 마이그레이션 전략을 수립해 단계별로 실행했습니다. 1단계로 개발 환경을 GCP에 먼저 구축하여 검증하고, 2단계로 QA 환경을 이전한 뒤, 3단계로 프로덕션 환경을 이전했습니다.

네트워크 설계는 Shared VPC (Host Project / Service Project 분리) 기반으로 구성하고, GitHub Actions 의 GCP 인증은 Workload Identity Federation 으로 전환하여 서비스 계정 키 발급/순환 부담을 제거하고 OIDC 기반 단명 토큰으로 대체했습니다. DNS 는 서브도메인 위임 (Hosting.kr → AWS Route53 → GCP Cloud DNS) 으로 사전 검증 단계를 거친 뒤, 최종 NS 변경 시점에만 점검 공고 후 cutover 를 수행했습니다. Filestore 의 SSD 최소 2.5TB 비용 문제는 Compute Engine + pd-balanced 1TB 디스크 기반 NFS 서버 자체 구축으로 우회하여 운영 비용을 최소화했습니다. Cloud Armor 로 region 차단 +

IP 화이트리스트 WAF(웹 방화벽) 정책을 구성하고, Cloud CDN 의 URL Map(LB 라우팅 규칙) 을 HTTP / HTTPS 두 개로 분리하여 HTTP → HTTPS 강제 리다이렉트와 정적 자산 캐싱을 함께 활성화했습니다.

Terraform 으로 GCP 인프라를 IaC 화 (IAM 제외 100%) 하고, 마이그레이션 완료 후 GitLab CI 와 ArgoCD 를 조합한 GitOps 기반 CI/CD 파이프라인을 구축했습니다. Helm Chart 로 환경별 설정을 표준화하고 Git 커밋 기반 배포 이력 관리 체계를 확립했습니다.

트러블슈팅

- AWS ALB 기반 라우팅을 GCP 글로벌 로드밸런서로 전환하는 과정에서, AWS ALB는 헬스체크 실패 백엔드에도 트래픽을 통과시키지만 GCP LB는 이를 차단하는 차이로 트래픽 라우팅 이슈 발생.
GCP NEG(Network Endpoint Group) 구조를 분석하여 GKE 워크로드에 맞는 헬스체크·백엔드 설정으로 재구성
- GKE Ingress + BackendConfig 의 healthcheck requestPath 응답이 누락되어 503 에러 발생. 해당 경로 응답을 보장하고 ManagedCertificate / FrontendConfig 와 정렬하여 외부 진입 트래픽 정상화
- Cloud CDN 마이그레이션 후 HTTP → HTTPS 리다이렉트 누락으로 게임 클라이언트 파일 다운로드가 실패.
URL Map 을 HTTP / HTTPS 두 개로 분리(`default_url_redirect.https_redirect=true`) 하고 HTTP forwarding rule 에 80 포트 target HTTP proxy 를 별도 매핑하여 해결
- DNS를 GCP Cloud DNS로 위임한 뒤 Google Search Console에 도메인이 자동 등록되지 않아 검색 노출·도메인 소유 확인에 문제 발생. Search Console이 발급한 verification TXT 레코드를 Cloud DNS에 추가해 도메인 소유권을 검증한 뒤 재등록하여 해결

성과

- 클라우드 비용 50% 절감 (월 \$10,000 → \$5,000), 연간 약 \$60,000 비용 절약
- 3단계 전략으로 전체 서비스 마이그레이션 완료. 리소스 복사 방식으로 다운타임 최소화하고, 최종 DNS 전환 시에는 게임 서비스 특성을 반영해 약 2시간 내외 점검 공지 후 전환
- 전체 GCP 인프라를 Terraform으로 IaC 관리 체계 구축 (TFLint/Checkov 정적 분석 및 Terratest 단위 테스트 도입으로 코드 품질 확보)
- 배포 시간 50% 단축 (40분 → 20분), 롤백 시간 95% 감소 (30분 → 1~2분)
- 주간 배포 횟수 3배 증가 (2회 → 6회), Git 커밋 기반 배포 이력 100% 추적 가능
- Workload Identity Federation 도입으로 GitHub Actions 의 GCP 서비스 계정 키 관리 부담을 제거하고 키 유출 리스크를 원천 차단, 단명 토큰 기반 보안 모델로 전환

기술 스택

GCP (GKE, Cloud SQL, Cloud Storage, VPC, Global LB 등), Shared VPC, Workload Identity Federation, Cloud Armor, Cloud CDN, Cloud DNS, NFS, Terraform, GitLab CI, ArgoCD, GitHub Actions, Kubernetes, Helm, Docker

프로젝트 2: 게임 데이터 수집 자동화 시스템 개발

기여도 100% (Python 스크립트 개발 및 운영) 3개월 (2024.01 ~ 2024.03)

문제

게임 및 블록체인 데이터를 수동으로 수집하고 있어 분석가가 매일 2~3시간을 데이터 수집에 소비했으며, 휴먼 에러로 인한 데이터 누락이 빈번하게 발생했습니다.

접근 전략

MongoDB · CloudSQL · Google Analytics · Dune 등 멀티 데이터 소스를 BigQuery 로 적재하고, 분석가용 Google Sheets 까지 자동 갱신되는 데이터 파이프라인을 GCP 서비스 조합으로 구축했습니다. MongoDB 는 Dataflow Flex Template 기반 ETL 로 BigQuery 에 적재하고, CloudSQL 은 BigQuery 의 직접 connection 기능을 활용하며, GA / Dune 은 각자의 API (Dune 은 별도 API Key) 로 호출하여 일관된 인덱스 스키마로 정규화했습니다.

Python Cloud Function 이 BigQuery 쿼리 결과를 Google Sheets API 로 적재하고, Cloud Scheduler 가 Daily (전일 데이터) / Monthly (전월 데이터) 두 가지 주기로 자동 트리거합니다. BigQuery 적재 후에는 별도 중복 제거 Cloud Function 이 정기 실행되어 데이터 정합성을 유지합니다. 전체 인프라 (BigQuery Dataset · Cloud Function · Scheduler · Cloud Storage · Artifact Registry · IAM 권한 · Service Account) 는 Terraform 으로 IaC 관리하여 환경 재구축과 변경 추적을 표준화했습니다.

트러블슈팅

- 파이프라인이 다수 GCP 서비스(BigQuery · Dataflow · Cloud Functions · Cloud Scheduler · Sheets · Drive 등)에 의존해, 신규 프로젝트 재구축 시 service API 미활성화로 배포·실행이 실패.
필요한 service API를 사전 enable하도록 표준화하여 재구축 시 누락 방지
- CloudSQL → BigQuery 직접 connection(federated query)이 리전 불일치와 connection Service Account 권한 부족으로 실패. connection을 동일 리전에 생성하고 connection SA에 Cloud SQL Client 권한을 부여하여 해결
- Dune API의 execute → poll → fetch 비동기 실행 구조와 rate limit으로 결과 누락·타임아웃 발생. 실행 상태 폴링 + 백오프로 결과 수신을 보장하고 API Key를 Secret Manager로 분리
- Daily 재실행 시 동일 행이 BigQuery에 재적재되는 비역등 문제 발생. 중복 제거 Cloud Function에 MERGE / ROW_NUMBER() 정합성 로직을 적용해 적재를 멍등화(재실행해도 결과 동일)하고 누락·중복 0 유지
- BigQuery 결과를 Google Sheets API로 적재할 때 단일 시트 1천만 셀 한도와 write 쿼터 초과로 갱신이 실패. values.batchUpdate 청크 분할과 exponential backoff, 시트 분할로 해결
- Google Sheets(Drive 계열) API의 분당 요청 횟수 한도로 다수 시트를 연속 갱신할 때 429 rate limit 발생. 분당 허용 횟수에 맞춰 호출 pacing(throttle) 과 배치 처리로 한도 내에서 동작하도록 구현

성과

- 데이터 수집 자동화 95% 달성 (일 2~3시간 수동 작업 제거)

- 자동화로 휴먼 에러 제거, 데이터 누락률 0% 달성
- 전체 인프라 Terraform IaC화로 동일 파이프라인을 다른 GCP 프로젝트에 30분 내 재구축 가능, 변경 이력 100% Git 추적

기술 스택

BigQuery, Dataflow, Cloud Functions, Cloud Scheduler, Cloud Storage, Artifact Registry, Terraform, Python, Google Sheets API, Dune API, MongoDB, CloudSQL, Docker

프로젝트 3: PLG Stack 기반 모니터링 시스템 구축

기여도 100% (시스템 설계 및 구축) 4개월 (2024.04 ~ 2024.07)

문제

Kubernetes 클러스터와 애플리케이션의 실시간 모니터링 체계가 없어 장애 발생 시 사후 대응만 가능했으며, 원인 파악에 과도한 시간이 소요되었습니다.

접근 전략

Prometheus로 메트릭을, Loki로 로그를 수집하여 Grafana 대시보드로 통합 시각화하는 PLG Stack을 구축했습니다. CPU·메모리·네트워크 사용률과 애플리케이션 로그를 실시간으로 모니터링하고, 임계치 초과 시 Slack 알림을 자동 발송하도록 구성했습니다.

성과

- 임계치 기반 알림으로 장애 사전 감지 체계 확립
- 실시간 모니터링 및 알림 체계로 장애 대응 시간 단축
- 서비스 가용성 향상
- SLI/SLO 수립 및 Alertmanager 알람 노이즈 최적화로 운영팀 피로도 감소 및 실질적 장애 감지 정확도 향상

기술 스택

Prometheus, Loki, Grafana, Promtail, Fluent-bit, Slack API

아이오차드

2022.04 ~ 2023.03

Infra Engineer

PaaS(Kubernetes + OpenStack + Ceph) 기반 인프라 솔루션을 고객사에 구축하고 기술 지원을 제공했습니다. 금융권·공공기관 온프레미스 클라우드 인프라를 설계·구축하고, 고객사 운영팀 교육을 직접 맡았습니다.

프로젝트 1: 고객사 맞춤형 PaaS 솔루션 구축

기여도 100% (인프라 설계 및 Kubernetes 클러스터 구축) 11개월 (2022.04 ~ 2023.03)

문제

고객사마다 요구하는 환경과 서버 스펙이 상이하여, 표준화된 솔루션으로는 대응이 어려웠습니다.

접근 전략

고객사 요구사항에 맞춰 Kubernetes 클러스터를 HA(High Availability) 구성으로 설계하고, OpenStack으로 가상머신 관리 환경을 구축했습니다. Ceph로 블록·파일·오브젝트 스토리지를 통합 제공하고, 고객사 운영팀에 인프라 운영 교육을 병행했습니다.

성과

- 5개 고객사에 PaaS 솔루션 성공적으로 구축 및 납품
- Kubernetes 클러스터 HA 구성으로 안정적 운영
- Ansible 기반 서버 프로비저닝 및 구성 관리 자동화로 고객사별 배포 시간 단축 및 환경 불일치(Configuration Drift) 이슈 0건 달성

기술 스택

Kubernetes, OpenStack, Ceph, Ansible, Rocky Linux/CentOS

프로젝트 2: DP사 Kubernetes 운영 교육 프로그램 설계·운영

기여도 100% (교육 설계 및 진행) 6개월 (2022.06 ~ 2022.11)

문제

DP사 운영팀이 Kubernetes·컨테이너 기반 인프라 운영 경험이 부족하여, PaaS 솔루션 도입 후 자체 운영에 어려움이 예상되었습니다.

접근 전략

Kubernetes 기초부터 클러스터 운영·트러블슈팅까지 단계별 교육 커리큘럼을 설계하고, 실습 환경을 구성해 실무 중심으로 운영했습니다. OpenStack·Ceph 스토리지 운영 교육도 병행해 전체 PaaS 스택 운영 역량을 확보하도록 도왔습니다.

성과

- 수강자 평균 10명 · 회당 교육 4시간 규모로 커리큘럼을 운영하여 DP사 운영팀의 Kubernetes 자체 운영 체계 확립

- 교육 완료 후 고객사 기술 문의 건수 감소, 자체 장애 대응 가능 수준 달성
- 교육 자료를 표준화하여 이후 타 고객사 교육에도 재 활용

기술 스택

Kubernetes, OpenStack, Ceph, Ansible, Rocky Linux/CentOS

프로젝트 3: Kubernetes 인증서 자동 갱신 프로세스 개선

기여도 100% (단독 수행, 전체 고객사 배포로 확산) 2주 (2022.11)

문제

Kubernetes 클러스터 인증서가 1년마다 만료되어 고객사마다 연간 약 5시간의 갱신 작업이 필요했으며, 만료 시 서비스 장애가 발생할 위험이 있었습니다.

접근 전략

Kubernetes 인증서 관리 프로세스를 분석하고, kubeadm 설정을 수정하여 인증서 유효기간을 1년에서 10년으로 연장했습니다. 기존 운영 중인 클러스터에도 적용 가능한 자동화 스크립트를 개발하여 전체 고객사에 배포했습니다.

트러블슈팅

- kubeadm 설정 수정으로 유효기간을 연장하는 방식은 당시 kubeadm으로 초기화한 클러스터 버전에만 적용 가능했고, v1.15.x / v1.16.x에서는 `kubeadm alpha certs renew` 에 알려진 버그가 있어 일부 인증서가 정상 갱신되지 않았습니다.
이 경우 kubeadm 명령에 의존하지 않고 `/etc/kubernetes` 백업 후 인증서 파일을 직접 10년으로 재생성하는 공개 오픈소스 스크립트([update-kube-cert](#))를 찾아 활용하고, etcd·apiserver·controller-manager·scheduler·kubelet 재시작 절차와 함께 적용해 버전에 관계없이 동작하도록 했습니다.

성과

- 인증서 갱신 주기 10배 연장 (연 1회 → 10년에 1회)
- 고객사 운영 부담 연간 약 50시간 절감 (고객사 10곳 기준)
- 인증서 만료로 인한 장애 리스크 제거

기술 스택

Kubernetes, kubeadm, Bash Script, OpenSSL